Manual Definitivo: Arquitetura de Visão e Áudio do Robô T800 (Integração com Servidor Externo)

Este documento é a versão estendida e aprofundada do guia de integração do robô humanoide T800. Ele foi elaborado para cobrir minuciosamente todos os detalhes técnicos, fluxos de dados, arquitetura de rede, comandos de terminal, código completo da ponte I/O e um vasto guia de solução de falhas (troubleshooting). O objetivo é garantir que a equipe de desenvolvimento consiga capturar o fluxo de vídeo da câmera e reproduzir áudio de forma estável, sem comprometer a segurança ou o equilíbrio do robô.

1. Arquitetura de Hardware e Isolamento de Processamento

Para garantir que as operações de Inteligência Artificial não causem travamentos no robô (o que poderia resultar em quedas e danos físicos graves), a arquitetura do T800 foi projetada com divisão de tarefas. O robô possui dois computadores de bordo principais [cite: 2]:

- **Nezha N97 (Intel Atom x7425E):** Responsável pelo controle de movimento e equilíbrio de baixo nível. Opera no IP 192.168.0.163 [cite: 2]. **Não deve ser utilizado para processamento de IA.**
- **NVIDIA AGX ORIN 64G:** O módulo de alta capacidade de cálculo (275 TOPS), rodando Ubuntu, projetado para aplicações avançadas e processamento de sensores. Opera no IP 192.168.0.162 [cite: 2, 4]. É neste módulo que a integração de câmera e áudio ocorrerá.

A estratégia de "Zero Impacto" consiste em usar o AGX Orin exclusivamente como um roteador de dados (dispositivo de I/O). Ele irá capturar o vídeo comprimido e despachá-lo para um **Servidor Backend Externo**, que fará o reconhecimento facial e devolverá o áudio [cite: 4].

2. Preparação da Rede e Acesso Remoto (SSH)

Antes de iniciar a configuração dos sensores, é essencial garantir a comunicação de rede perfeita com o módulo Orin.

2.1. Configuração da Sub-rede

A rede interna do robô opera no padrão 192.168.0.0/24 [cite: 2]. O seu computador de

desenvolvimento deve estar nesta mesma sub-rede. Conecte-se via ponto de acesso Wi-Fi do robô ou por cabo Ethernet (porta RJ45 Gigabit Ethernet) [cite: 2].

2.2. Acesso ao Terminal do Orin

Abra o terminal e execute a conexão SSH. O usuário e a senha de fábrica são ambos ubuntu.

```
# Conexão padrão
ssh ubuntu@192.168.0.162

# Conexão com suporte a interface gráfica (X11 Forwarding)
ssh -X ubuntu@192.168.0.162
```

Possível Falha: Se a conexão falhar (Timeout), verifique se o seu firewall não está bloqueando portas ou se o IP da sua máquina não está conflitando. Garanta que a sua máquina recebeu um IP na faixa 192.168.0.x.

3. Integração de Vídeo: Câmera Odin2 (T800 -> Servidor)

O T800 é equipado com sensores visuais avançados localizados na cabeça [cite: 2]. A câmera Odin2 é conectada diretamente ao Orin (cujo IP interno de fábrica para a câmera é 192.168.100.251) [cite: 4]. O objetivo aqui é extrair o vídeo comprimido e enviá-lo ao servidor externo.

3.1. Compreendendo os Tópicos e Formatos do ROS2

O driver oficial da câmera publica dados em tópicos ROS2. Para não sobrecarregar a rede sem fio e o processador, **nunca assine o tópico de imagem crua (raw)** para envio externo. Utilize sempre o tópico de imagem comprimida.

- **Tópico Alvo Absoluto:** /manifold/odin/device0/camera0/compressed (Câmera Esquerda) [cite: 4].
- **Tipo de Mensagem:** sensor_msgs/CompressedImage [cite: 4].
- **Formato Recomendado:** mjpeg (Compressão JPEG, menor consumo de banda) [cite: 4].

3.2. Iniciando o Driver da Câmera

Após acessar o Orin via SSH, carregue o ambiente ROS2 e inicie o driver com os parâmetros corretos para otimizar o envio de rede:

```
source /opt/ros/humble/setup.bash
ros2 launch odin_ros_driver_rev1 driver.launch.py \
 image_width:=640 \
 image_height:=544 \
 image_fps:=30 \
 image_format:=mjpeg \
 start_rviz:=false
```

4. Integração de Áudio e Fala (Servidor -> T800)

A resposta do seu servidor deve ser um áudio sintetizado. O T800 possui 2 alto-falantes de alta fidelidade no tronco e uma matriz de 4 microfones na cabeça [cite: 2]. O sistema de áudio **não utiliza tópicos ROS2**. Ele se comunica diretamente com o barramento do sistema operacional Linux (ALSA/PulseAudio).

4.1. Configuração do Ambiente de Áudio

No terminal do Orin, instale as dependências nativas e as bibliotecas Python necessárias (incluindo o Flask para o servidor local):

```
sudo apt-get update
sudo apt-get install -y libasound2-dev portaudio19-dev
pip3 install flask requests sounddevice soundfile numpy
```

5. Implementação da Ponte de I/O em Python (Script Completo)

O código a seguir implementa a arquitetura de "Zero Impacto" da EngineAI. Ele separa a recepção assíncrona de áudio em uma thread com o Flask e mantém o nó do ROS2 dedicado exclusivamente a despachar os frames da câmera com o prefixo absoluto correto.

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import CompressedImage
import requests
import threading
from flask import Flask, request
import sounddevice as sd
import soundfile as sf
import io

# ==========================================
# CONFIGURAÇÕES DE REDE
# ==========================================
# Substitua pelo IP real do seu servidor na mesma rede (ex:
192.168.0.x)
BACKEND_URL =
"http://IP_DO_SEU_SERVIDOR:5000/api/reconhecimento"

ORIN_AUDIO_PORT = 8080

app = Flask(__name__)

# ==========================================
# 1. SERVIDOR DE ÁUDIO (Servidor -> T800)
# ==========================================
@app.route('/play_audio', methods=['POST'])
def play_audio():
    try:
        audio_bytes = request.data
        data, fs = sf.read(io.BytesIO(audio_bytes))
```

```
        sd.play(data, fs)
        sd.wait()

        return "Áudio reproduzido com sucesso", 200
    except Exception as e:
        return f"Erro ao reproduzir áudio: {str(e)}", 500

def start_audio_server():
    app.run(host='0.0.0.0', port=ORIN_AUDIO_PORT, threaded=True)

# ========================================
# 2. NÓ ROS2 DE VISÃO (T800 -> Servidor)
# ========================================
class CameraBridgeNode(Node):
    def __init__(self):
        super().__init__('camera_bridge_node')

        # O prefixo padrão oficial da câmera Odin2 no T800
        topic_name = '/manifold/odin/device0/camera0/compressed'

        self.subscription = self.create_subscription(
            CompressedImage,
            topic_name,
            self.image_callback,
            10
        )
        self.get_logger().info(f"Ponte iniciada. Enviando frames para:
{BACKEND_URL}")
        self.get_logger().info(f"Escutando áudios na porta local:
```

```
{ORIN_AUDIO_PORT}")

    def image_callback(self, msg):
        jpeg_bytes = bytes(msg.data)

        try:
            response = requests.post(
                BACKEND_URL,
                data=jpeg_bytes,
                headers={'Content-Type': 'image/jpeg'},
                timeout=0.2
            )

            if response.status_code != 200:
                self.get_logger().warning(f"Aviso do backend:
{response.status_code}")

        except requests.exceptions.RequestException as e:
            self.get_logger().error(f"Falha de conexão com backend: {e}")

# ========================================
# FUNÇÃO PRINCIPAL
# ========================================
def main(args=None):
    audio_thread = threading.Thread(target=start_audio_server,
daemon=True)
    audio_thread.start()

    rclpy.init(args=args)
    camera_bridge = CameraBridgeNode()
```

```
    try:
        rclpy.spin(camera_bridge)
    except KeyboardInterrupt:
        print("\nDesligando a ponte de I/O do T800...")
    finally:
        camera_bridge.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Como executar isso no T800 (NVIDIA Orin)

Para colocar esse sistema no ar, você precisará conectar-se via SSH ao Orin e iniciar primeiro o driver da câmera, e depois o seu script:

1. **Passo 1: Iniciar o Driver da Câmera Odin2 (Terminal 1)**
   Acesse o Orin via SSH (ssh ubuntu@192.168.0.162). O driver da câmera precisa estar rodando para gerar as imagens comprimidas JPEG. Inicie-o forçando a resolução ideal para rede, como 640x544:
   ```
   # Carrega o ambiente do ROS2
   source install/setup.bash

   # Inicia a câmera publicando em MJPEG a 30 FPS
   ros2 launch odin_ros_driver_rev1 driver.launch.py image_width:=640
   image_height:=544 image_fps:=30 image_format:=mjpeg
   start_rviz:=false
   ```

2. **Passo 2: Rodar o seu Script de Ponte (Terminal 2)**
   Abra uma segunda conexão SSH com o Orin. Salve o seu código (por exemplo, como ponte_io.py) e execute-o com o ambiente ROS2 carregado:
   ```
   source install/setup.bash
   python3 ponte_io.py
   ```

Pronto! Seu script agora estará capturando a imagem comprimida em altíssima velocidade e enviando ao servidor. O servidor processa o rosto, monta o arquivo de áudio e dispara um POST para http://192.168.0.162:8080/play_audio. O T800 tocará o som instantaneamente em seus alto-falantes de alta fidelidade no peito.

6. Solução de Problemas, Possíveis Falhas e Dúvidas Frequentes

Sintoma / Falha	Causa Raiz	Solução Recomendada
Conexão SSH recusada ou lenta.	Conflito de IP ou perda de sinal da rede 192.168.0.0/24.	Verifique se está usando um cabo Ethernet ou se o Wi-Fi está estável. Faça um ping para 192.168.0.162.
Interferência nos Tópicos ROS2.	Vários computadores na mesma rede rodando ROS2 com o mesmo DOMAIN_ID.	Defina um ID isolado no terminal do Orin: export ROS_DOMAIN_ID=1.
Driver da Câmera não encontra dispositivo.	Firewall bloqueando pacotes UDP (portas 60000-60010).	Desative o firewall (ufw) ou libere as portas. Em último caso, force o IP no launch: device_ip:=192.168.100.251.
Resolução ou FPS não se aplicam.	A câmera não suporta a configuração informada no firmware atual.	Verifique os logs de inicialização do driver ("Sensor capabilities query") e use apenas as combinações suportadas.

Sintoma / Falha	Causa Raiz	Solução Recomendada
Perda de pacotes (Vídeo com atraso).	O protocolo DDS/UDP do ROS2 está usando a política BEST_EFFORT.	Implemente um arquivo de QoS (QoS override) definindo a confiabilidade para RELIABLE.
Script Python falha ao tocar áudio.	O usuário ubuntu não possui permissão no hardware ALSA.	Adicione o usuário ao grupo de áudio: sudo usermod -aG audio ubuntu e reinicie.
Sobrecarga de CPU no Orin e travamentos.	Tentativa de processar imagens cruas (Raw) no Orin.	Assine apenas o tópico compactado e delegue a decodificação para o backend externo.

7. Considerações de Segurança

De acordo com o manual do T800, é fundamental manter precauções rigorosas ao testar esses sistemas:

- O robô pesa aproximadamente 84,5 kg. Mantenha um raio de segurança de 5 metros livres de pessoas e obstáculos.
- Durante o desenvolvimento do software, caso o robô apresente anomalias (ruídos ou movimentos instáveis), utilize imediatamente as combinações de botões do controle para acionar a **Parada de Emergência** (Volume + e Volume - simultaneamente, ou os botões Back e Start).
- A parada de emergência fará o robô perder o suporte instantaneamente. Portanto, utilize ganchos ou suporte de suspensão durante a fase de testes na bancada.